

System Diagram

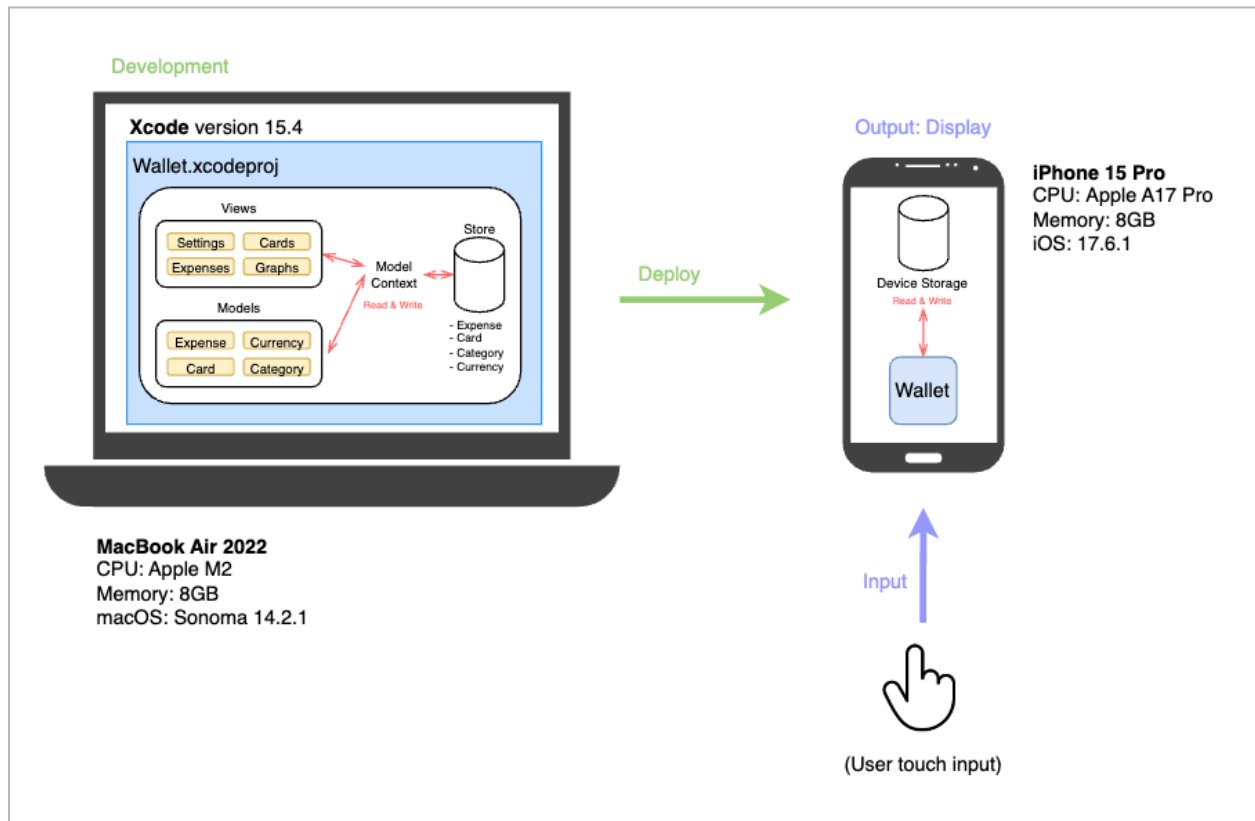


Fig. 1: System diagram of project. The application is developed on a MacBook, and the project files are then used to deploy the application product on an iPhone.

Flow Diagrams

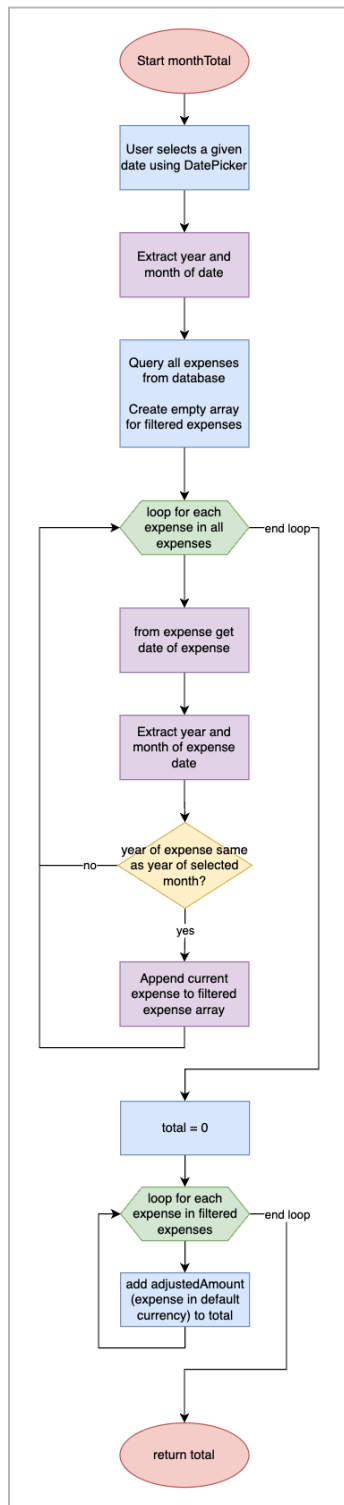


Fig. 2: Flow diagram of the function to find the total sum of expenditures in a user selected month.

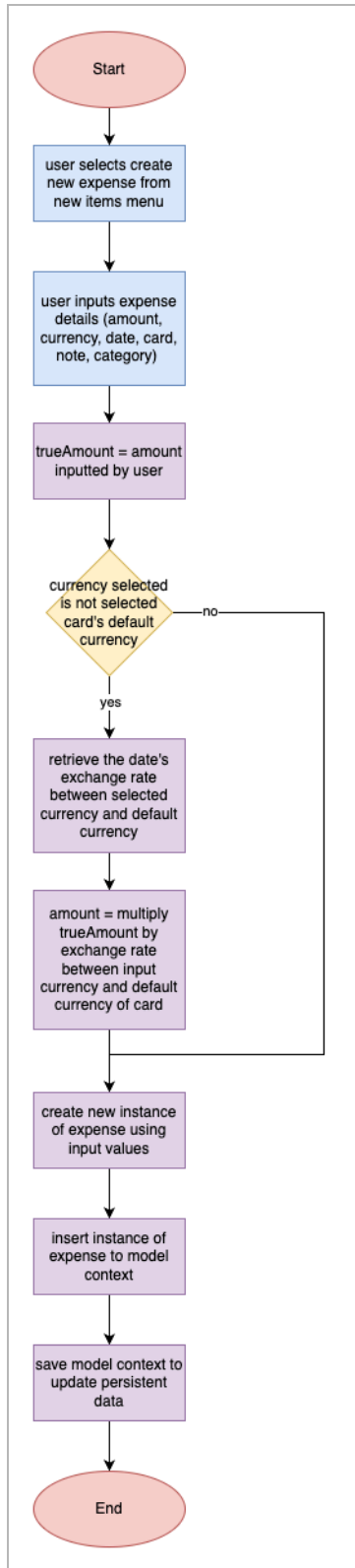


Fig. 3: Flow diagram of the create new expense function.

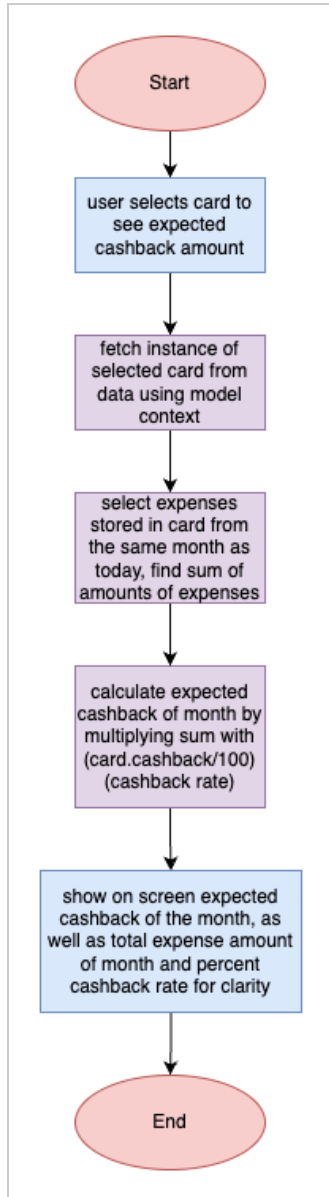


Fig. 4: Flow diagram of calculation done to obtain expected cashback of a user selected card.

UML Diagram

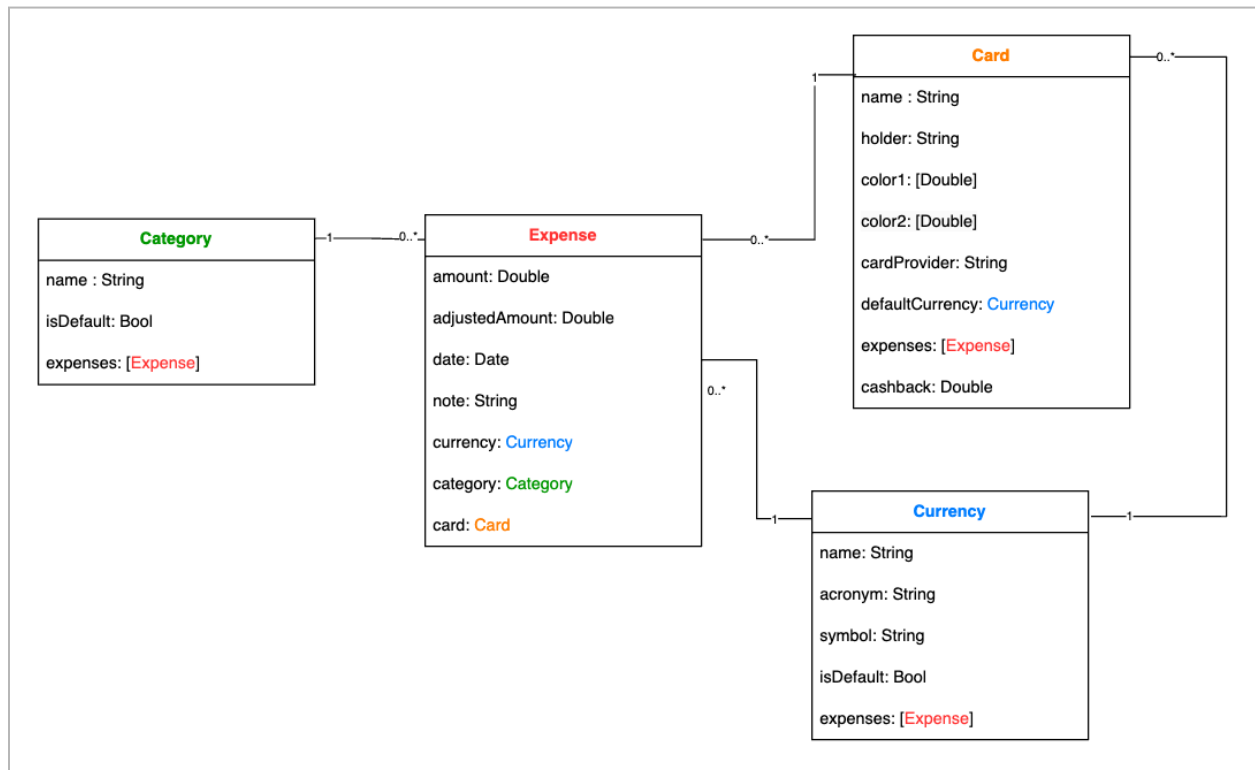


Fig. 5: UML diagram showing the model classes and their relationships, used to store persistent data in SwiftData.

Test Plan

Test Content	Success Criteria Tested	Intended Input	Example User Input	Expected Output
Ability to create credit card objects and store in application	#1	From create new card menu, user inputs strings such as card nickname, card owner, cashback rate. User selects from preexisting choices such as default currency, other visual components.	Name: "Savings" Card owner: "Bob Sanders" Default currency: "USD" Cashback rate: 0.5(%)	Instance of the creditCard class with attributes inputted/selected by user stored in persistent storage via model context. This can be fetched and hence displayed in UI.
Ability to create expenses and store in application	#2	From create new expense menu, user inputs strings such as expenditure amount, notes. User selects preexisting choices such as date, currency, category.	Amount: 100 Date: 2024/10/1 Category: "Food" Card: Savings Currency: "USD" Note: "Bought groceries"	Instance of the expense class with attributes inputted/selected by user stored in persistent storage via model context. This can be fetched and hence displayed in UI.
Ability to edit expenses	#3	Go to details page of a given expense. Edit button allows user to re-input strings and re-select choices.	Amount: 80 Date: 2024/10/1 Category: "Food" Card: Daily Currency: "USD" Note: "Used different card for discount"	Upon submission of edits, attributes of existing expense object is changed in persistent storage.
Ability to delete expenses	#3	User selects a given expense, pressing button allows for confirmation of deletion.	N/A	Upon confirmation of deletion, existing expense object is removed from persistent storage.
Ability to display monthly statistics by card	#4	Go to the details page of a given card, user selects a given date using DatePicker.	Card: Savings Date: 2024/10/21	Basic statistics such as total spent in the month of selected date (September 2024 in example case) on selected

				card is displayed in UI.
Ability to display cumulative statistics by month	#4	Go to the all expenses page, user selects a given date using DatePicker.	Card: Daily Date: 2024/10/15	Basic statistics such as total spent in the month of selected date (October 2024 in example case) is displayed in UI.
Ability to generate a pie chart of monthly breakdown of categories by card	#5	Go to the details page of a given card, user selects a given date using DatePicker.	Card: Savings Date: 2024/10/17	App filters expenses to those made on given card, during selected month (August 2024 in example case), sorts into amount sum by category to generate a pie chart to display on UI.
Ability to generate a pie chart of monthly cumulative breakdown of categories	#5	Go to the graph view page, user selects a given date using DatePicker.	Date: 2024/10/20	App filters expenses to those during selected month (July 2024 in example case), sorts into amount sum by category to generate a pie chart to display on UI.
Ability to perform currency exchanges	#6	When creating a new expense, user selects a currency different from the default currency.	defaultCurrency: "USD" Expense: Amount: 100 Currency: "JPY" Date: 2024/10/30	Automatic conversion of amount using conversion rate retrieved using API request, stored as attribute adjustedAmount of an expense object in persistent storage.

Table 1: Table of test plan outlining general flow of intended user inputs to test final product against success criteria.